EAST WEST UNIVERSITY

Department of Computer Science and Engineering

Project Report

on

TYE — Track Your Expenses

Course Code	CSE 412
Course Title	Software Engineering
Section	02
Submitted To	Dr. Mohammad Mahdi Hassan

Submitted By: Team NewToDev (Team 17)

Student ID	Name	Role
2023-2-60-117	MD. Sifat Bin Islam	Team Lead
2023-2-60-176	Md. Arifur Rahman Shihab	Backend Developer
2023-2-60-294	Md. Sabbir Ahmed	Testing
2023-2-60-217	Adnan Anan Sikder	Frontend Developer

Submission Date: 19 March 2026

Table of Contents

Table of Contents 2
Chapter 1: Project Introduction 5
1.1 Introduction 5
1.2 Overview of the System 5
1.3 Scope and Objectives 6
1.4 Stakeholders 7
Chapter 2: Requirements 8
2.1 Requirements Collection Process 8
2.2 User Stories 8
2.3 Acceptance Criteria 9
US-01: Google Sign-In 9
US-02: Log a Transaction 9
US-03: Upload Receipt for OCR 10
US-04: Auto-Categorisation 10
US-07 and US-08: Monthly Budget and Alerts 10
2.4 Use Case Diagram 11
Chapter 3: Architecture and Detailed Design 12
3.1 System Architecture 12
3.1.1 Presentation Layer 12
3.1.2 Application Layer 12
3.1.3 Data Layer 12
3.2 Technology Stack 13
Chapter 4: Database Design 14
4.1 Overview 14
4.2 Entity Relationship Description 14
4.3 Table Definitions 14
4.4 Indexing Strategy 15
4.5 Entity Relationship Diagram 16
Chapter 5: Class Design 17
5.1 Overview 17
5.2 Backend Classes and Modules 17
5.2.1 Database Model Classes (models.py) 17
5.2.2 Pydantic Schema Classes (schemas.py) 17
5.2.3 OCR Service Module (ocr_service.py) 17
5.2.4 Parser Service Module (parser_service.py) 18

5.3 Frontend Components and Hooks 19
5.3.1 App.jsx 19
5.3.2 useApi.js (Custom Hook) 19
5.3.3 Page and Layout Components 19
5.4 Class Diagram 20
Chapter 6: Design Patterns 21
6.1 Overview 21
6.2 Factory Method Pattern (Creational) 21
Purpose 21
Application in the TYE Project 21
6.3 Adapter Pattern (Structural) 21
Purpose 21
Application in the TYE Project 21
Chapter 7: UI/GUI Design 23
7.1 Design Philosophy 23
7.2 Typography 23
7.3 Landing Page 23
7.4 Login Page 24
7.5 Dashboard 24
Sidebar 24
Overview Panel 24
Transactions Panel 24
Budgets Panel 24
Analytics and Report Panels 24
7.6 Responsive Design 25
7.7 UI Diagrams and Screenshots 25
Chapter 8: Unit Testing 31
8.1 Testing Strategy 31
8.2 Backend Test Coverage 31
Schema Validation Tests (test_schemas.py) 31
API Route Tests (test_api_routes.py) 31
8.3 Frontend Test Coverage 31
Component and Hook Tests (App.test.jsx) 31
8.4 Test Summary 32
Appendix 33
A. Project Resources 33

Chapter 1: Project Introduction

1.1 Introduction

Managing personal finances requires consistent effort, particularly in tracking day-to-day expenditures across multiple spending categories. For individuals in Bangladesh, this challenge is intensified by the widespread use of cash transactions alongside digital payment platforms such as bKash and Nagad, which often leave users without a consolidated view of their spending. Standard bank statements, where available, typically list transactions without contextual category information, making it difficult for users to understand their spending patterns in a meaningful way.

TYE (Track Your Expenses) is a web-based personal finance management application developed to address this problem. The system enables users to log, categorise, and analyse their daily expenditures through an interactive web interface. A key component of the system is its integration with the Google Cloud Vision API, which allows users to upload photographs of physical or handwritten receipts and have the relevant financial information — including the transaction amount, date, and merchant name — extracted automatically. This reduces the manual effort required for data entry and supports users who deal with handwritten or printed receipts in both Bengali and English.

Beyond data entry, TYE provides a structured budget management system that allows users to define monthly spending limits for individual expenditure categories. The system tracks actual spending against these limits in real time and issues threshold-based alerts before users exceed their budgeted amounts. A visual analytics dashboard presents spending trends through graphical summaries, giving users an immediate understanding of their financial behaviour across a given month.

The application is built using a React-based single-page application for the client side and a Python FastAPI backend for server-side logic. Authentication is handled through Google OAuth 2.0, which eliminates the need for a traditional username-and-password system. Data is persisted in a PostgreSQL relational database. This report documents the full development of TYE, covering the system's requirements, architecture, database and class design, design patterns applied, and the user interface.

1.2 Overview of the System

TYE follows a three-tier web application architecture consisting of a presentation layer, an application layer, and a data layer. The presentation layer is a React-based single-page application that renders the landing page, login interface, and the main dashboard. The dashboard is organised into functional tabs covering transactions, budgets, savings goals, analytics, and monthly reports.

The application layer is a FastAPI server that exposes a RESTful API. It handles Google OAuth 2.0 token verification, transaction and budget CRUD operations, OCR processing via the Google Cloud Vision API, receipt text parsing, monthly summary computation, and report generation. The data layer consists of a PostgreSQL database with three primary tables: users, transactions, and budgets.

The OCR pipeline operates as follows. A user uploads a photograph of a receipt through the dashboard. The backend saves the file and calls the Google Cloud Vision API using the DOCUMENT_TEXT_DETECTION feature, which is optimised for structured documents and supports both printed and handwritten text in Bengali and English. The raw text returned by the API is then passed to a custom parser module that uses regular expressions and keyword matching to extract the transaction amount, date, and merchant name. The detected spending category is determined by matching merchant and item keywords against a predefined category taxonomy.

1.3 Scope and Objectives

The scope of the TYE project encompasses the following functional areas:

- User authentication via Google OAuth 2.0.
- Manual and OCR-assisted transaction logging.
- Monthly budget creation and threshold-based alert management.
- Savings goal tracking with user-defined targets.
- Visual spending analytics through charts and category breakdowns.
- Monthly expense reporting with category-level comparisons.
- Support for Bengali and English text in OCR processing.

The following objectives guided the development process:

- To reduce the manual effort required for expense logging by automating data extraction from receipt photographs.
- To support bilingual OCR processing for Bengali and English receipts, addressing the local context of the intended user base.
- To provide a budget management mechanism that notifies users when expenditure approaches or exceeds defined category limits.
- To present spending data through a visual dashboard that allows users to identify patterns in their financial behaviour.
- To implement a secure and stateless authentication mechanism using Google OAuth 2.0.

1.4 Stakeholders

The following stakeholders are associated with the TYE project:

Primary Users	Students, salaried professionals, and small business owners who wish to monitor and manage their personal expenditures.
Client	Rafat Bin Islam, Staff Engineer, Agoda — the project sponsor who defined the initial requirements and reviewed deliverables.
Development Team	Team NewToDev (Team 17) — a four-member team responsible for the design, implementation, and testing of the system.
Academic Supervisor	Dr. Mohammad Mahdi Hassan, Department of Computer Science and Engineering, East West University.

Chapter 2: Requirements

2.1 Requirements Collection Process

Requirements for TYE were gathered through a combination of client interviews, direct user observation, and an analysis of comparable personal finance tools available in the Bangladeshi market. The client, Rafat Bin Islam, defined the core problem domain: users who make cash and digital payments lack a convenient way to track and categorise their expenditure, particularly when receipts are handwritten or written in Bengali.

The development team conducted structured interviews with representatives from the intended user groups — university students, salaried employees, and small business owners. The interviews focused on identifying the most common pain points in manual expense tracking and understanding the frequency and nature of receipt-based transactions. Additionally, the team reviewed existing applications such as Wallet, Money Manager, and Monefy to identify functional gaps relevant to the local context.

Requirements were prioritised using the MoSCoW method (Must Have, Should Have, Could Have, Won't Have). High-priority requirements were those directly supporting the core expense logging and budget management workflows. Medium-priority requirements addressed reporting and analytics. Lower-priority requirements, such as data export and advanced filtering, were deferred to a subsequent development iteration.

2.2 User Stories

The following user stories define the functional requirements of the TYE system. Each story is expressed from the perspective of an end user and is accompanied by a priority classification.

ID	Priority	Title	Description
US-01	High	Google Sign-In	As a new user, I want to sign in using my Google account so that I can access the application without creating a separate account or remembering a new password.
US-02	High	Log a Transaction	As a registered user, I want to manually enter an expense by specifying the amount, category, description, and date so that I can record transactions that do not have an associated receipt.
US-03	High	Upload Receipt for OCR	As a user, I want to upload a photograph of a receipt so that the system can automatically extract the amount, date, and merchant name without requiring me to type the information manually.
US-04	High	Auto-Categorisation	As a user, I want uploaded or manually entered transactions to be automatically assigned a spending category (for example, Food or Transport) based on the merchant or description so that I do not need to tag each item individually.
US-05	High	Edit Transaction Details	As a user, I want to edit the amount, category, description, or date of any existing transaction so that I can correct inaccuracies introduced by OCR or by my own input.

ID	Priority	Title	Description
US-06	High	Delete a Transaction	As a user, I want to delete a transaction record so that I can remove incorrectly entered or duplicate entries.
US-07	High	Set Monthly Budget	As a budget-conscious user, I want to assign a monthly spending limit to a specific category so that the system can monitor my expenditure against that limit.
US-08	High	Budget Alert Threshold	As a user, I want to receive a visual alert when my spending in a category approaches or exceeds the defined monthly limit so that I can take corrective action before overspending.
US-09	Medium	View Spending Dashboard	As a user, I want to view a summary of my current month's expenditure, including the total amount spent and a breakdown by category, presented as charts and statistics on the main dashboard.
US-10	Medium	Monthly Analytics Report	As a user, I want to view a month-by-month comparison of my spending so that I can identify trends and changes in my financial behaviour over time.
US-11	Medium	Savings Goal Tracking	As a user, I want to define a savings target with a name and a target amount so that I can track my progress towards a financial goal.
US-12	Low	Custom Expense Categories	As a user, I want to create custom spending categories beyond the default set so that I can classify expenses specific to my personal or professional context.

2.3 Acceptance Criteria

The following acceptance criteria define the conditions under which the primary high-priority user stories are considered complete.

US-01: Google Sign-In

- The login page presents a Google Sign-In button rendered by the @react-oauth/google library.
- Upon successful Google authentication, the backend verifies the received ID token using the Google OAuth 2.0 library and either creates a new user record or updates the existing record in the database.
- The server returns the user's identifier, name, email address, and profile picture URL to the client.
- The user's session is stored in sessionStorage and is cleared automatically when the browser tab is closed.
- An appropriate error message is displayed if authentication fails.

US-02: Log a Transaction

- The transaction form accepts a numeric amount, a category selected from a dropdown, a text description, and a date field.

- The system rejects submissions where the amount is zero or negative and displays a validation error.
- On submission, a POST request is sent to the /transactions/ endpoint and the new record is persisted in the database.
- The transaction list and dashboard statistics update immediately upon successful creation.

US-03: Upload Receipt for OCR

- The system accepts image files in JPEG, PNG, and WebP formats with a maximum size of 10 MB.
- The uploaded image is sent to the Google Cloud Vision API using the DOCUMENT_TEXT_DETECTION feature with Bengali and English language hints.
- The extracted text is stored against the transaction record and presented to the user in a review interface.
- If the OCR service is unavailable or returns an error, a descriptive error message is shown to the user.

US-04: Auto-Categorisation

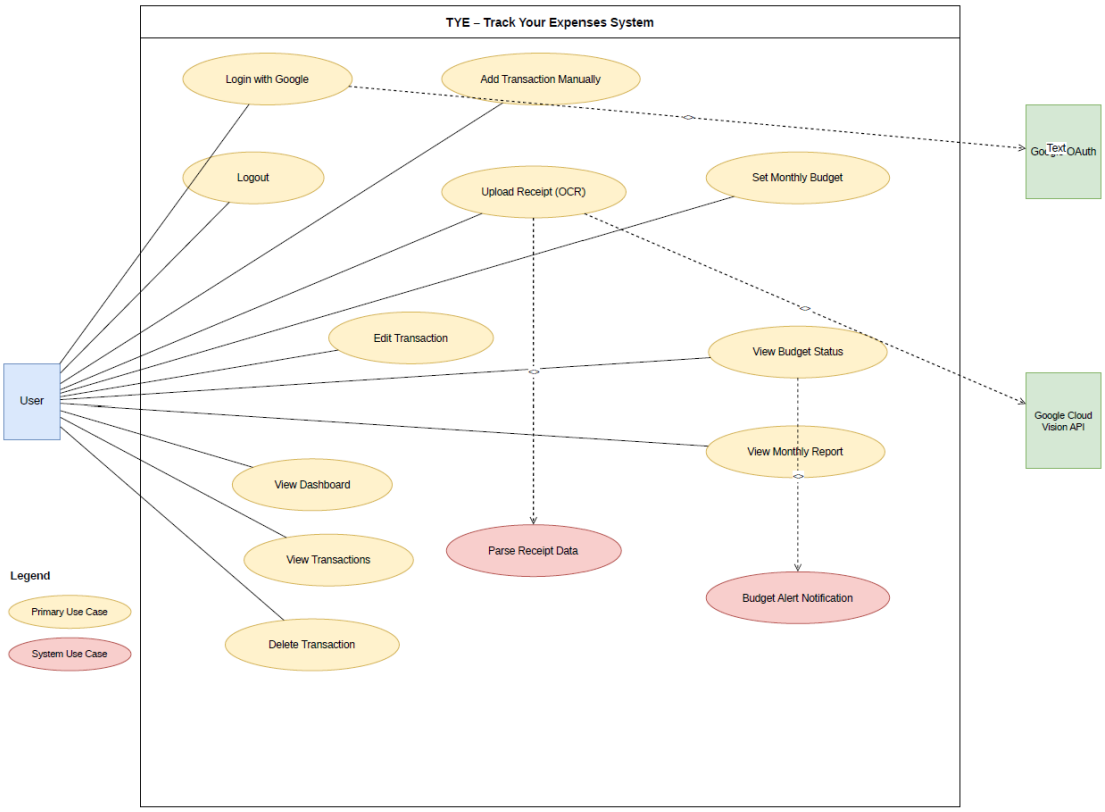
- The parser module analyses the extracted or entered text and matches it against a predefined keyword taxonomy covering the categories: Food, Transport, Shopping, Utilities, Entertainment, and Rent.
- If a matching keyword is identified, the corresponding category is assigned automatically.
- If no keyword match is found, the category defaults to Other.
- A confidence score between 0 and 1 is computed for each assigned category.

US-07 and US-08: Monthly Budget and Alerts

- The budget creation form requires a category, a monthly limit (as a positive numeric value), and a month in YYYY-MM format.
- The system prevents the creation of a duplicate budget for the same user, category, and month combination.
- The budget status endpoint computes the total spending for the specified category and month and returns the percentage of the budget consumed.
- A visual warning indicator is displayed on the budget progress bar when spending reaches or exceeds the user-defined alert threshold (default: 80%).
- A distinct over-budget indicator is displayed when spending exceeds 100% of the defined limit.

2.4 Use Case Diagram

The use case diagram for TYE identifies the interactions between the two primary actors — the Registered User and the System Administrator — and the functional use cases of the system.



Chapter 3: Architecture and Detailed Design

3.1 System Architecture

TYE is structured according to a three-tier client-server architecture. This model separates the system into a presentation layer, an application layer, and a data layer, each with clearly defined responsibilities. This separation promotes maintainability, supports independent scaling of individual tiers, and reduces coupling between system components.

3.1.1 Presentation Layer

The presentation layer is a React single-page application built with the Vite toolchain. The application comprises three primary views: the Landing Page, the Login Page, and the Dashboard. Navigation between views is managed through component-level state within App.jsx, using a page state variable rather than a client-side router, which simplifies the routing model for this application's scope.

The Dashboard is further divided into functional panels rendered conditionally based on the active tab selection. These panels include: Overview (summary statistics and charts), Transactions (full transaction list with add and edit capabilities), Budgets (monthly budget management), Savings (savings goal tracking), Analytics (month-by-month expenditure trends), and Report (monthly detailed reports). A background particle animation canvas is rendered persistently behind all views to provide visual consistency across the application.

3.1.2 Application Layer

The application layer is implemented using the Python FastAPI framework. The server exposes a RESTful API organised into the following groups of endpoints:

- Authentication endpoints: /auth/google (POST) and /auth/verify/{user_id} (GET).
- Transaction endpoints: /transactions/ (POST), /transactions/{user_id} (GET), /transactions/{id} (PUT, DELETE), and /summary/{user_id} (GET).
- Budget endpoints: /budgets/ (POST), /budgets/{user_id}/{month} (GET), /budgets/{id} (PUT, DELETE), and /budget-status/{user_id}/{month} (GET).
- OCR and parsing endpoints: /parse-image (POST), /upload-receipt/{transaction_id} (POST), /extract-text/{transaction_id} (POST), and /parse-receipt/{transaction_id} (POST).
- Report endpoint: /monthly-report/{user_id} (GET).

Cross-origin resource sharing (CORS) is configured to permit requests from the Vite development server. Static file serving is mounted at /uploads to allow the client to retrieve uploaded receipt images directly.

3.1.3 Data Layer

The data layer consists of a PostgreSQL relational database accessed through the SQLAlchemy ORM. The database engine and session factory are configured in database.py. Database sessions are injected into FastAPI endpoints via a dependency provider function (get_db), which ensures that sessions are created and closed cleanly for each request.

3.2 Technology Stack

Layer	Technology	Justification
Frontend	React 18 + Vite	Component-based UI with fast development server and HMR support.
Frontend State	React useState / useCallback	Lightweight state management sufficient for single-user session scope.
HTTP Client	Axios	Promise-based HTTP library with clean error handling.
Backend	Python FastAPI	Asynchronous, type-annotated API framework with automatic OpenAPI documentation.
ORM	SQLAlchemy	Mature Python ORM with support for PostgreSQL and in-memory SQLite for testing.
Database	PostgreSQL	Robust relational database with full ACID compliance and indexing support.
Authentication	Google OAuth 2.0	Eliminates password management; verifies identity using Google's token infrastructure.
OCR	Google Cloud Vision API	DOCUMENT_TEXT_DETECTION supports handwritten Bengali and English with high accuracy.
Validation	Pydantic	Declarative data validation integrated with FastAPI for request schema enforcement.
Testing	pytest + Vitest	pytest for backend unit and integration tests; Vitest and React Testing Library for frontend.

Chapter 4: Database Design

4.1 Overview

The TYE database is implemented in PostgreSQL and managed through the SQLAlchemy ORM. The schema consists of three tables: users, transactions, and budgets. These tables are defined as Python model classes in models.py, and SQLAlchemy's create_all function generates the corresponding database schema on application startup. The use of an ORM layer ensures that database interactions are expressed as Python object operations, which reduces the risk of SQL injection vulnerabilities and improves code readability.

4.2 Entity Relationship Description

The data model reflects a one-to-many relationship between users and both transactions and budgets. A single user may have zero or more transaction records and zero or more budget records. Referential integrity is enforced through foreign key constraints: the user_id column in both the transactions and budgets tables references the id column of the users table. Cascade deletion behaviour is not configured at the ORM level in the current version; records must be deleted explicitly through the API.

4.3 Table Definitions

Table 4.1: users

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented, indexed.
name	String	Full display name obtained from Google OAuth token. Not null.
email	String	Unique, indexed. Used as a secondary identifier for the user.
google_id	String	Unique, indexed. The subject identifier (sub) from the Google ID token. Not null.
picture	String	URL of the user's Google profile photograph. Nullable.

Table 4.2: transactions

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented.
amount	Float	Transaction value in Bangladeshi Taka. Must be greater than 0 (enforced at the API layer).
category	String	Spending category label (e.g., Food, Transport). Not null.

Column	Data Type	Constraints / Notes
description	String	Free-text description of the transaction (e.g., merchant name). Nullable.
date	String	Transaction date stored as a string in YYYY-MM-DD format.
image_path	String	Relative path to the uploaded receipt image file. Nullable.
raw_text	String	Full OCR-extracted text from the receipt image. Nullable.
user_id	Integer	Foreign key referencing users.id. Associates the record with its owner.

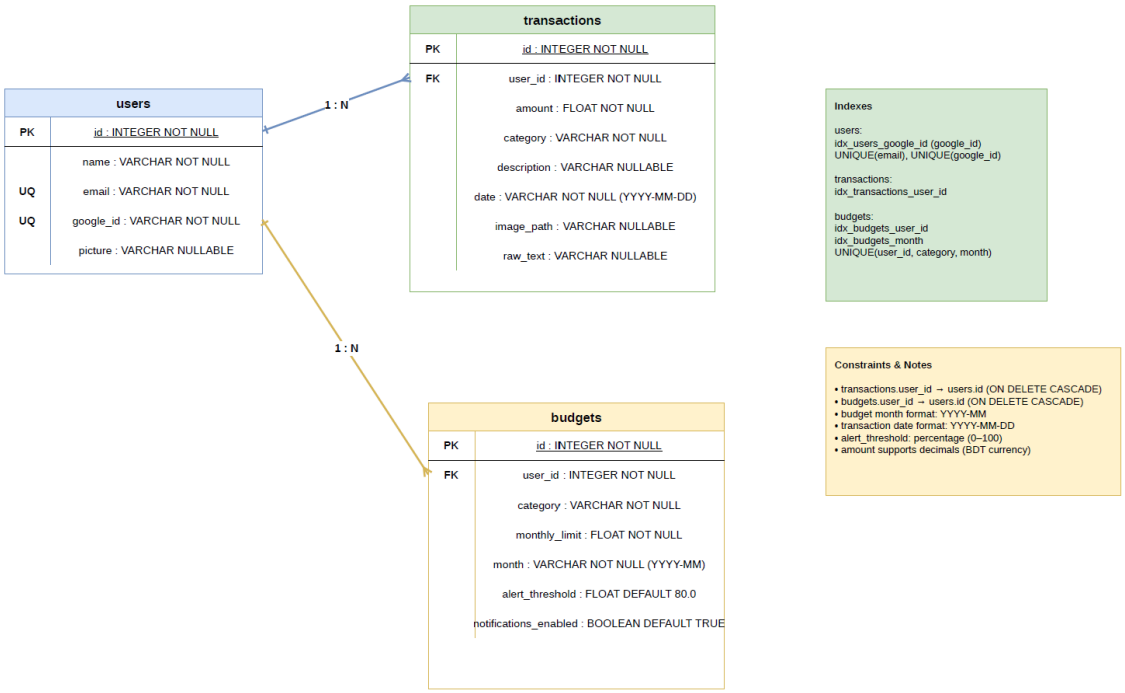
Table 4.3: budgets

Column	Data Type	Constraints / Notes
id	Integer	Primary key, auto-incremented.
user_id	Integer	Foreign key referencing users.id. Not null.
category	String	The expenditure category to which the budget applies. Not null.
monthly_limit	Float	The maximum permitted spending for this category in the given month.
month	String	The budget period expressed as YYYY-MM (e.g., 2026-03).
alert_threshold	Float	Percentage of the monthly limit at which a warning is triggered. Default: 80.0.
notifications_enabled	Boolean	Controls whether the threshold alert is active for this budget. Default: True.

4.4 Indexing Strategy

The users table has indexes on the email and google_id columns, both of which are used as lookup keys during authentication. The id columns across all three tables are indexed as primary keys. No additional composite indexes have been defined in the current version, as query volumes are expected to remain moderate within the initial deployment scope.

4.5 Entity Relationship Diagram



Chapter 5: Class Design

5.1 Overview

The class design of TYE is described across four functional layers: database model classes, Pydantic schema classes, service modules, and React component/hook units. The following sections describe the responsibilities and key members of each primary class or module. A UML class diagram is provided in the appendix.

5.2 Backend Classes and Modules

5.2.1 Database Model Classes (models.py)

The models.py module defines three SQLAlchemy declarative model classes. Each class maps directly to a database table and defines its columns using SQLAlchemy's Column construct.

- User: Represents an authenticated user. Key attributes include id, name, email, google_id, and picture. A bidirectional one-to-many relationship is defined with the Transaction class via the transactions back-reference.
- Budget: Represents a monthly spending limit for a specific category. Key attributes include user_id (foreign key), category, monthly_limit, month, alert_threshold, and notifications_enabled.
- Transaction: Represents a single expenditure record. Key attributes include user_id (foreign key), amount, category, description, date, image_path, and raw_text. The image_path and raw_text fields support the OCR pipeline.

5.2.2 Pydantic Schema Classes (schemas.py)

Pydantic schema classes enforce input validation and define the shape of API request and response bodies. The key schema classes are as follows:

- GoogleTokenRequest: Accepts a single string field (token) representing the Google ID token sent by the frontend after authentication.
- GoogleLoginResponse: The response schema returned after successful authentication, containing user_id, name, email, picture, and is_new_user.
- TransactionCreate: Validates the body of a transaction creation request. Requires amount (Float), category, description, date, and user_id. The image_path field is optional.
- TransactionUpdate: Used for partial updates. All fields (amount, category, description, date) are Optional, allowing callers to update only the fields that require modification.
- BudgetCreate: Validates budget creation requests. Requires category, monthly_limit, and month. The alert_threshold field defaults to 80.0 and notifications_enabled defaults to True.
- BudgetOut and TransactionOut: Response schemas that define what data is returned to the client after successful create or retrieve operations, using the from_attributes configuration for ORM compatibility.

5.2.3 OCR Service Module (ocr_service.py)

The ocr_service module encapsulates all interaction with the Google Cloud Vision API. Its public interface consists of three functions:

- is_paddleocr_available(): Returns a boolean indicating whether the API key is configured. Called by the application layer before executing any OCR operation.
- extract_text_from_image(image_path): Accepts a local file path, encodes the image in Base64, constructs the Vision API payload, executes the HTTP POST request, and returns the extracted text as a plain string. Raises specific exceptions for file-not-found, missing API key, timeout, and API error conditions.
- extract_text_with_details(image_path, min_confidence): Returns a dictionary containing the full extracted text, a list of per-word confidence scores, and aggregate statistics. Used for diagnostic and debugging purposes.
- get_ocr_info(): Returns metadata about the OCR configuration, including the engine name, supported languages, availability status, and pricing information.

The GoogleVisionConfig inner class holds the API key, language hints (["bn", "en"]), and the REST endpoint URL. This class-based configuration structure supports straightforward substitution of the API key through environment variables.

5.2.4 Parser Service Module (parser_service.py)

The parser_service module provides functions for extracting structured financial data from raw OCR text. The module supports both Bengali and English input through a preliminary digit normalisation step.

- normalize_digits(text): Converts Bengali numeral characters (০১২৩৪৫৬৭৮৯) to their ASCII equivalents so that subsequent regular expressions operate uniformly on all input.
- extract_amount(text): Applies a set of prioritised regular expression patterns — covering Bengali total keywords (মোট, সর্বমোট), Taka symbols (৳), English total keywords, and generic decimal number formats — to extract the primary transaction amount and an associated confidence score.
- extract_date(text): Applies patterns for DD/MM/YYYY, YYYY-MM-DD, and English month-name formats, normalising Bengali digits first. Returns the date in ISO 8601 format (YYYY-MM-DD) with a confidence score.
- extract_merchant(text): Examines the first four lines of the receipt text, filters out phone numbers, address-like lines, and purely numeric lines, and returns the first plausible merchant name with a confidence score.
- detect_category(text): Matches the text against a keyword taxonomy containing Bengali and English terms for six categories: Food, Transport, Shopping, Utilities, Entertainment, and Rent. Returns the category with the highest keyword match count and a confidence score.
- parse_receipt(text): The primary function called by the API endpoint. Calls the four extraction functions above and assembles their results into a single dictionary with amount, date, merchant, category, and confidence sub-keys.

5.3 Frontend Components and Hooks

5.3.1 App.jsx

App.jsx is the root component of the React application. It maintains a page state variable (landing, login, or dashboard) and a user state variable representing the currently authenticated user. These states are initialised from sessionStorage on component mount, enabling session persistence across page refreshes within the same browser tab. The component renders the appropriate page component based on the current page state and passes callback functions (onLogin, onLogout, onGetStarted, onBack) to child components.

5.3.2 useApi.js (Custom Hook)

The useApi hook is the data access layer for the Dashboard. It accepts the authenticated user's identifier and exposes the following state variables and functions to consuming components:

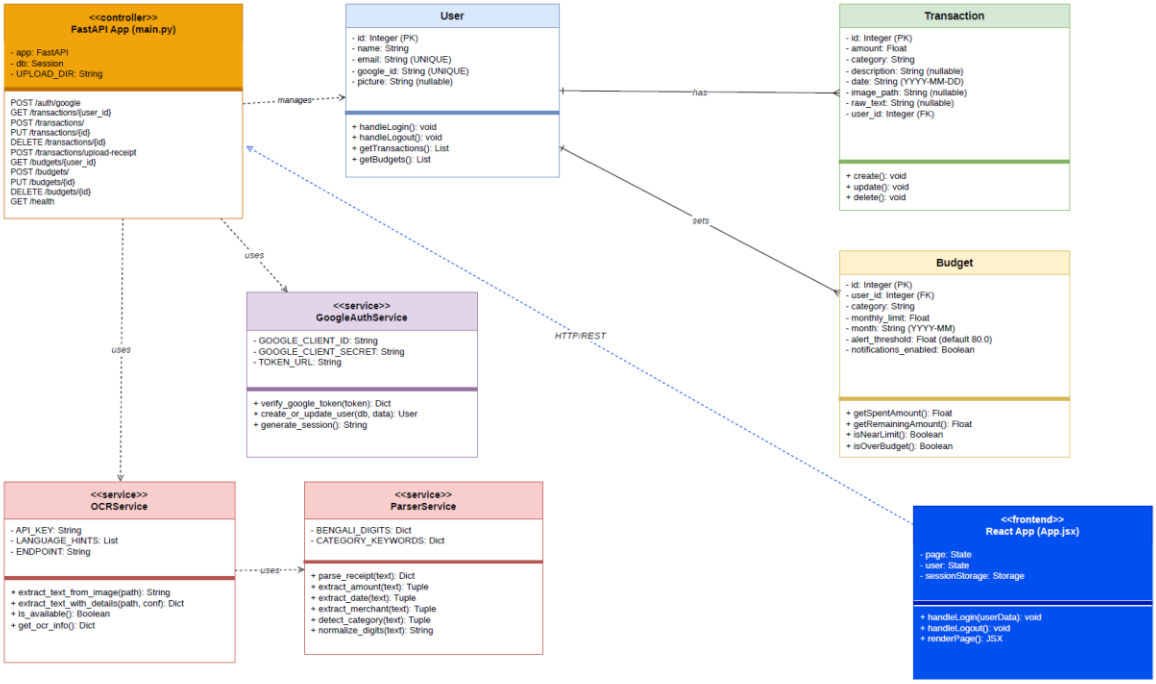
- State: transactions, budgets, summary, loading, savings, and cats (categories).
- Fetchers: fetchTransactions, fetchSummary, fetchBudgets, and fetchMonthlyReport, each making the corresponding Axios GET request.
- Mutations: addTransaction, deleteTransaction, updateTransaction, createBudget, deleteBudget, and updateBudget, each making the corresponding Axios POST, PUT, or DELETE request and then calling the relevant fetcher to refresh the local state.
- Utility: toYM(date) converts a JavaScript Date object to a YYYY-MM string, used to construct the month parameter for summary and budget API calls.

The savings and cats state variables are persisted to sessionStorage rather than the database, as no backend endpoints were implemented for these features in the current version.

5.3.3 Page and Layout Components

- LandingPage.jsx: Renders the marketing landing page with a navigation bar, hero section, feature cards, a how-it-works walkthrough, user testimonials, and a call-to-action section. Uses the IntersectionObserver API to trigger scroll-based animations.
- LoginPage.jsx: Renders the authentication interface using the GoogleOAuthProvider and GoogleLogin components from the @react-oauth/google library. On successful Google sign-in, sends the credential token to the backend and calls the onLogin callback with the returned user data.
- Dashboard.jsx: The primary application view post-authentication. Renders the sidebar navigation, top bar, tab navigation, and the active functional panel. Consumes the useApi hook for all data operations.
- ParticleCanvas.jsx: Renders an animated particle system on a persistent HTML5 canvas element positioned behind all page content. Uses requestAnimationFrame for continuous animation and handles window resize events.

5.4 Class Diagram



Chapter 6: Design Patterns

6.1 Overview

The TYE system applies two established software design patterns from the Gang of Four catalogue: the Factory Method pattern and the Adapter pattern. These patterns were selected to address specific structural challenges in the codebase — namely, database session lifecycle management and the integration of an external third-party API.

6.2 Factory Method Pattern (Creational)

Purpose

The Factory Method pattern provides an interface for creating objects in a superclass but allows subclasses or frameworks to alter the type of objects that will be created. Its primary objective is to encapsulate the instantiation logic, ensuring that objects are created with a consistent configuration without exposing the complex initialisation code to the client components that utilise them.

Application in the TYE Project

In the TYE system, the Factory Method pattern is utilised for database session management. Within the database.py module, the SQLAlchemy sessionmaker acts as the factory. Instead of having each FastAPI endpoint independently instantiate a database connection with hardcoded configurations, the system defines a SessionLocal factory.

This factory configuration is further abstracted by the get_db() dependency generator. When a client component (such as the transaction creation endpoint in main.py) requires database access, it calls upon this factory method. The factory handles the creation, configuration (e.g., setting autocommit=False and autoflush=False), and subsequent yielding of the Session object. This separation of concerns ensures that the application layer remains decoupled from the specific instantiation requirements of the data persistence layer.

6.3 Adapter Pattern (Structural)

Purpose

The Adapter pattern functions as a structural bridge between two incompatible interfaces. It involves creating an intermediary component that translates requests from a client into a format that a distinct, often third-party, service can process. Its purpose is to allow disparate classes or external systems to work together without requiring modifications to their existing source code.

Application in the TYE Project

The TYE architecture implements the Adapter pattern within the ocr_service.py module to integrate the external Google Cloud Vision API. The external API possesses a rigid and complex interface: it requires image data to be encoded in Base64 format and mandates a highly nested JSON payload structure (specifying features like DOCUMENT_TEXT_DETECTION and language hints). Furthermore, the external service returns a complex, deeply nested JSON response.

The internal application, however, merely requires a simple textual representation of the receipt. The ocr_service.py module acts as the Adapter by providing a simplified interface, specifically the extract_text_from_image(image_path) function. This function accepts a standard local file path, internally handles the Base64 encoding and payload structuring, executes the HTTP request, and parses the nested JSON response to extract the relevant text string. Consequently, the core application (in main.py) remains entirely isolated from the specific structural requirements of the Google Cloud Vision API, allowing for seamless integration.

Chapter 7: UI/GUI Design

7.1 Design Philosophy

The TYE user interface is designed around the principle of clarity and minimal cognitive load. Users engage with the application to understand their financial behaviour; the interface should therefore present data in a structured and unambiguous manner without drawing excessive attention to itself. A dark colour theme was selected as the primary visual language, using a deep navy background ('#030c23') with a gold accent colour ('#e8b84b') for emphasis and interactive elements. This palette was chosen to maintain sufficient contrast for readability while establishing a professional visual identity.

A global CSS variable system defined in global.css provides a single source of truth for the colour palette, typography, and spacing constants used across all components. This approach ensures visual consistency across the landing page, login interface, and dashboard without duplication of style definitions.

7.2 Typography

Three typefaces are used across the interface. Playfair Display (serif) is applied to headings and display text where editorial weight is required. Inter (sans-serif) serves as the primary body typeface for all instructional and data text, providing legibility at small sizes. Space Mono (monospace) is used for numeric values, dates, category labels, and other data-dense fields, which benefits from the fixed-width character spacing of a monospace font.

7.3 Landing Page

The landing page is the first point of contact for prospective users. It is structured as a single scrollable document comprising the following sections:

- Navigation Bar: Fixed at the top of the viewport, contains the TYE logo, navigation links (Features, How It Works, Reviews), and two call-to-action buttons (Log In and Get Started). The navigation bar becomes opaque with a backdrop blur effect when the user scrolls past the top of the page.
- Hero Section: Contains the primary value proposition headline, a supporting description, two call-to-action buttons, and a three-dimensional CSS-transformed mock dashboard card illustrating the application's output. Three floating badge elements animate continuously to reinforce the key user benefits.
- Statistics Band: Displays four headline statistics (active users, total tracked expenditure, satisfaction rate, and pricing) in a horizontally distributed layout.
- Features Section: Presents six feature cards in a responsive grid, each containing an icon, feature title, description, and a categorical tag.
- How It Works Section: A two-column layout presenting a four-step process walkthrough alongside a mock transaction list screen.
- Reviews Section: Three user testimonial cards presented in a responsive grid.
- Call-to-Action Section: A final conversion section with a prominent button directing users to the login interface.

7.4 Login Page

The login page is a two-panel layout. The left panel presents animated floating information cards displaying sample financial statistics and a user testimonial quote. A grid overlay and three radial gradient orb effects create visual depth without requiring image assets. The right panel contains the authentication form, which consists of the TYE title, a brief description, and the Google Sign-In button rendered by the @react-oauth/google library. Three security badge labels (Encrypted, OAuth 2.0, GDPR Compliant) are displayed below the authentication button to establish user trust. Error messages from failed authentication attempts are displayed inline above the security badges.

7.5 Dashboard

The dashboard is the primary working interface of the application. It adopts a two-column layout consisting of a persistent left sidebar and a main content area.

Sidebar

The sidebar displays the TYE brand logo, a grouped navigation menu, and a footer section showing the authenticated user's profile photograph, name, email address, and a logout button. Navigation items are organised into two groups: Main (Overview, Transactions, Budgets) and Insights (Savings, Analytics, Report). The active navigation item is highlighted using a gold tint background and a gold-coloured border.

Overview Panel

The overview panel presents four statistical cards at the top: total spending for the current month, total number of transactions, average transaction value, and the number of budget categories in use. Below the statistics, a pie chart visualises the current month's spending breakdown by category, and a bar chart displays the transaction totals for the seven most recent days. The five most recent transactions are listed beneath the charts for quick reference.

Transactions Panel

The transactions panel presents the complete transaction history for the authenticated user in a scrollable list. Each transaction item displays a category icon, the transaction description, a category tag, the date, and the amount. Inline edit and delete controls are provided for each item. An Add Transaction form is accessible directly within the panel, containing fields for amount, category, description, and date, along with an optional receipt image upload control.

Budgets Panel

The budgets panel lists all budgets defined for the current month, each displayed as a progress bar showing the ratio of actual spending to the defined monthly limit. The progress bar changes colour based on the spending threshold: green for normal, orange when the alert threshold is reached, and red when the limit is exceeded. A budget creation form is embedded in the panel, requiring the user to specify the category, monthly limit, and optional alert threshold percentage.

Analytics and Report Panels

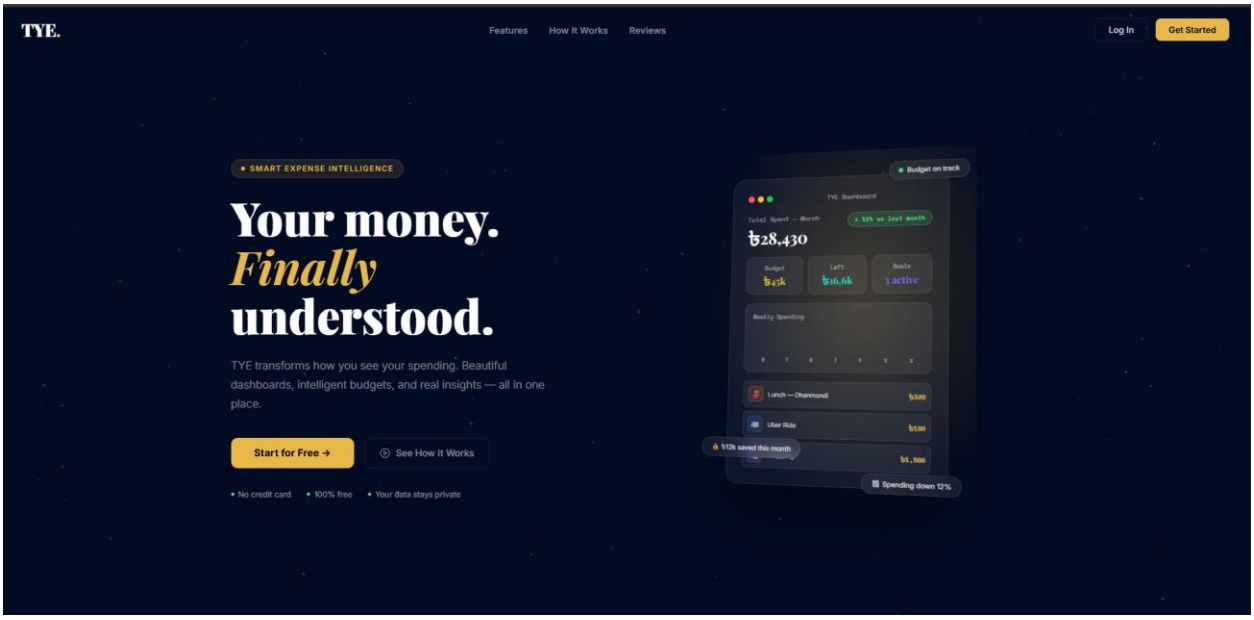
The analytics panel presents a grid of charts covering monthly spending trends, category distribution, and comparative statistics across months. The report panel provides a month selector

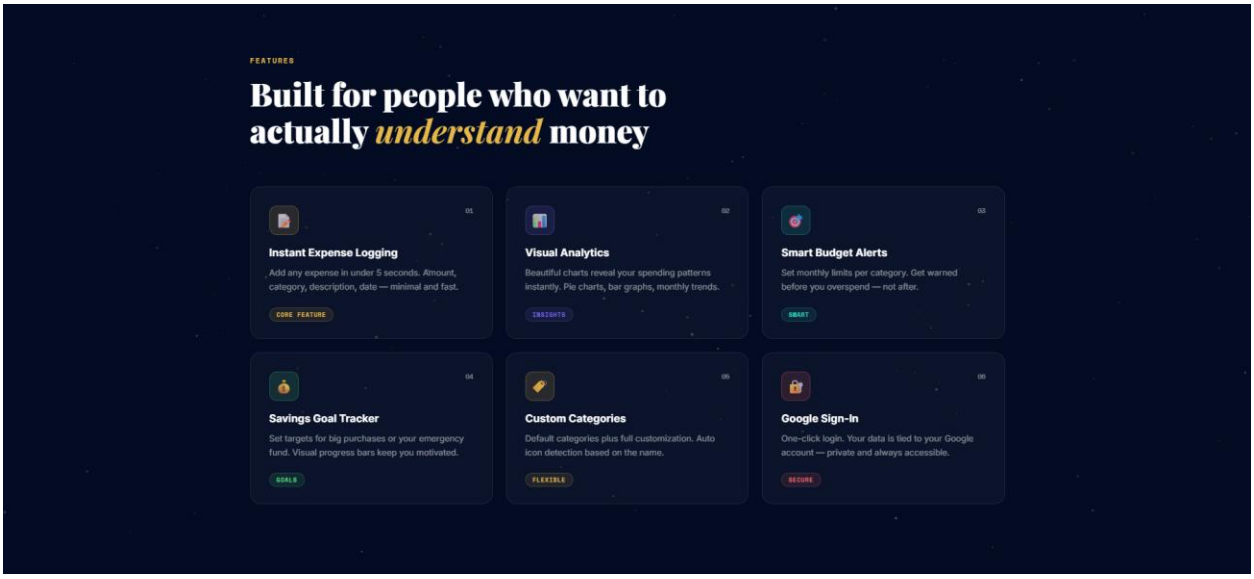
control and displays a detailed category breakdown for the selected month, including budget versus actual spending figures where budgets are defined.

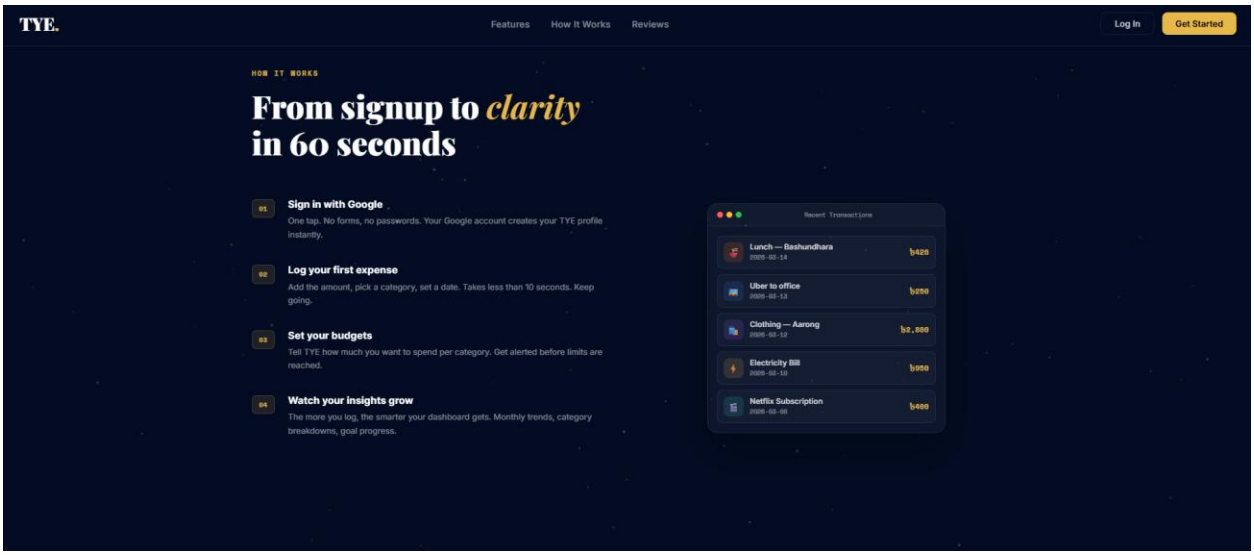
7.6 Responsive Design

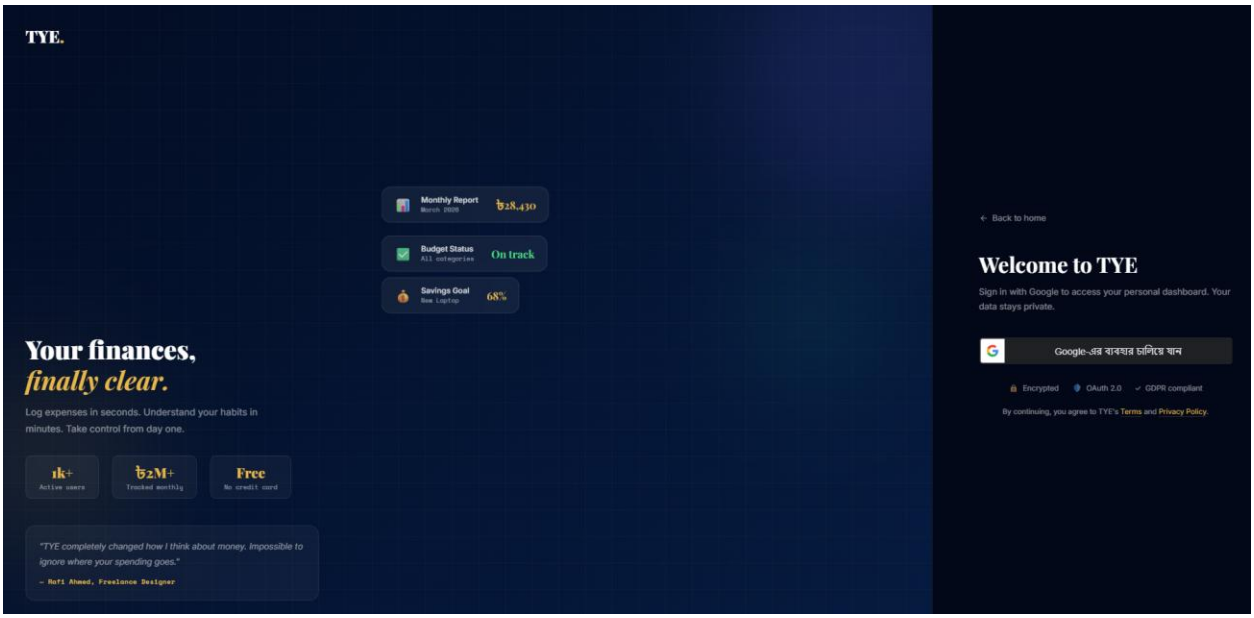
Media queries defined in Dashboard.css and LandingPage.css apply layout adjustments for screens narrower than 900 pixels. At this breakpoint, the sidebar is hidden on the dashboard, and multi-column grid layouts collapse to single-column arrangements. On the landing page, the hero section's side-by-side layout stacks vertically, and the mock dashboard card is hidden to preserve space. The login page similarly restructures from a horizontal two-panel layout to a vertically stacked arrangement at the same breakpoint.

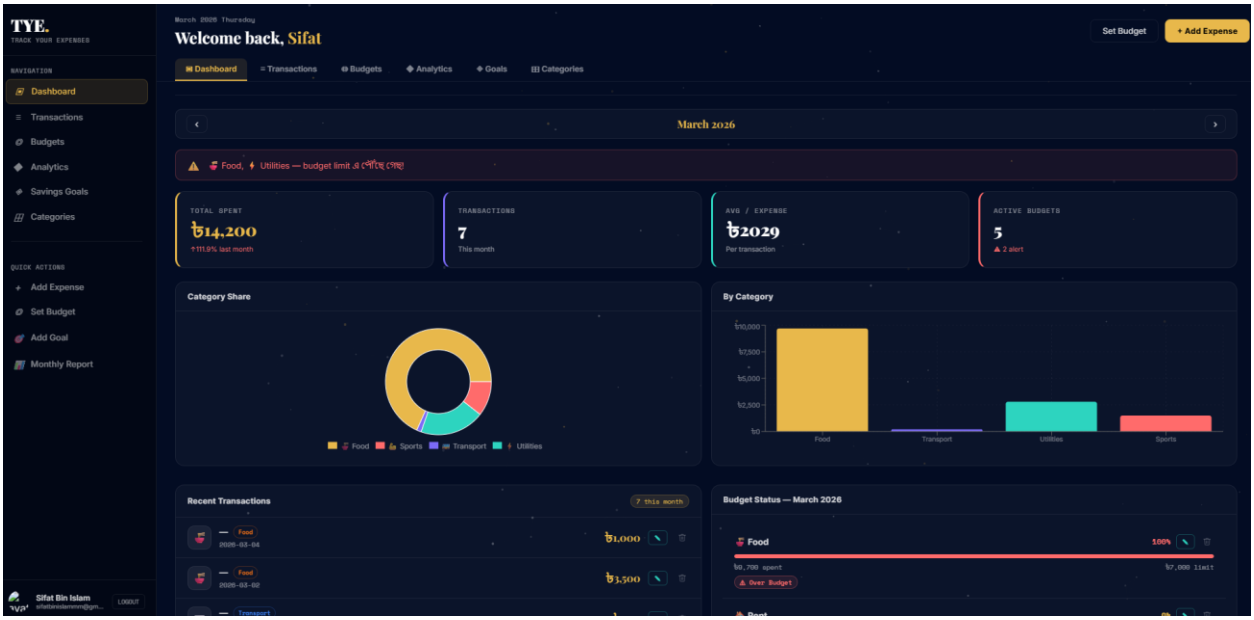
7.7 UI Diagrams and Screenshots

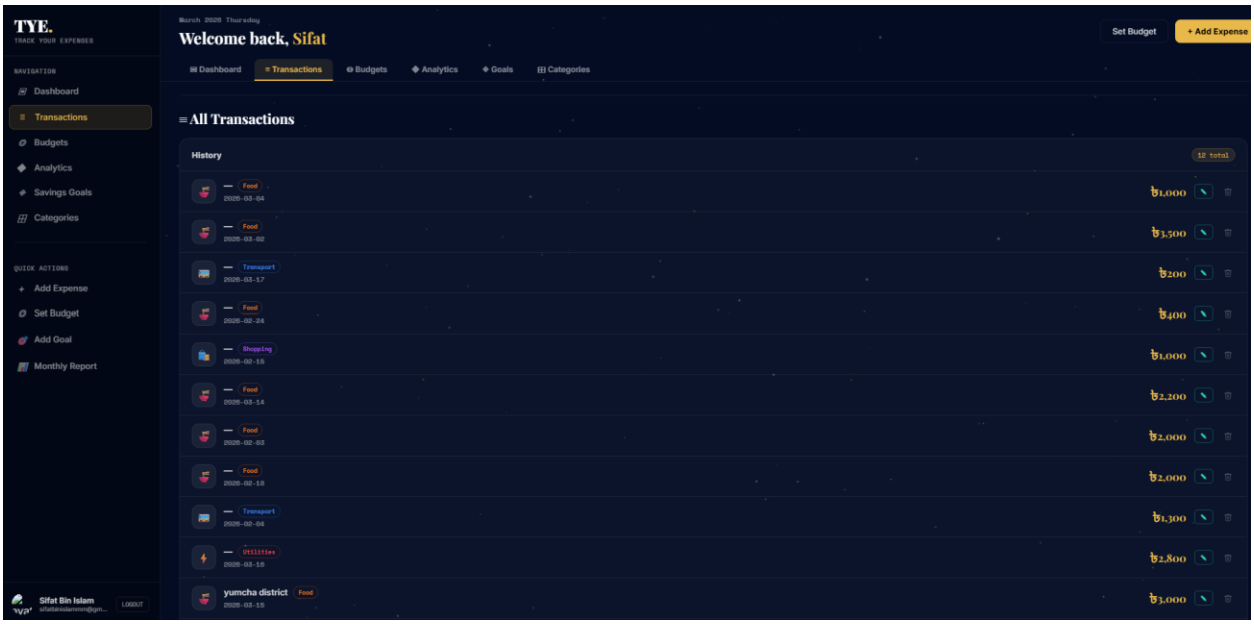


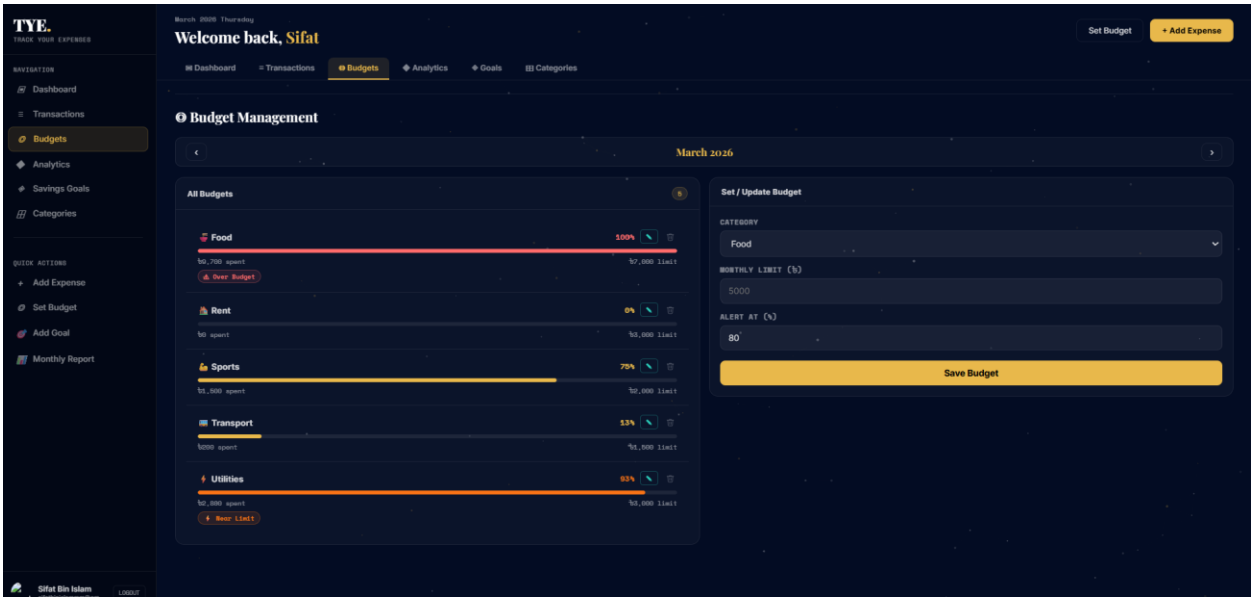


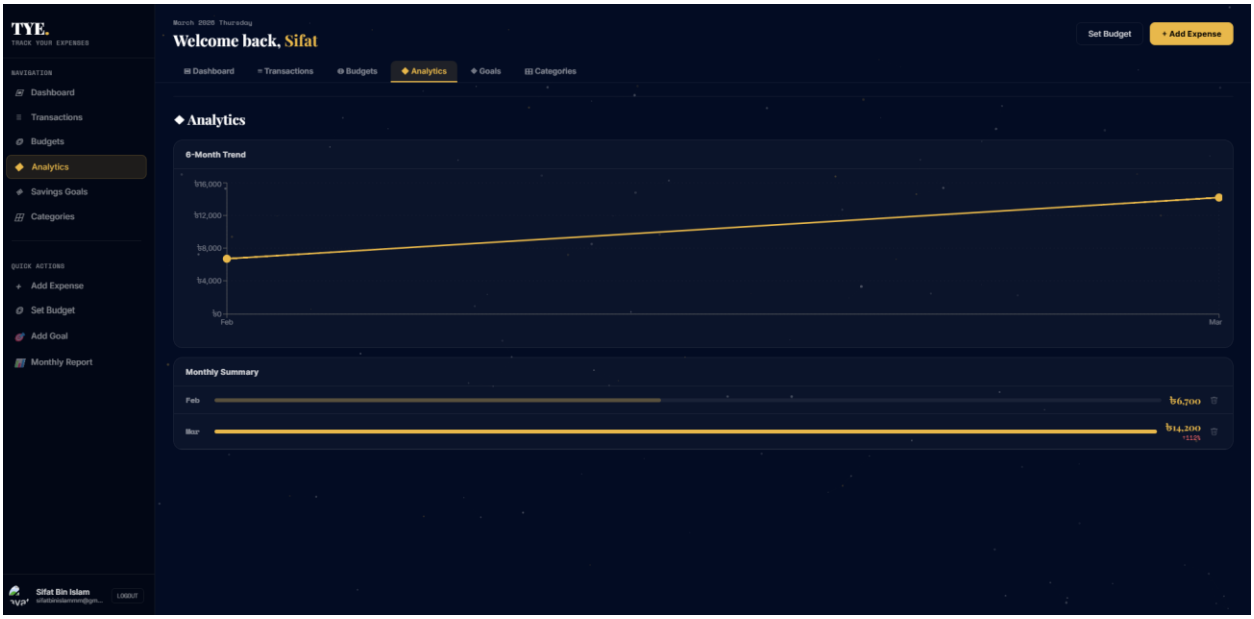

TYE.
Features
How It Works
Reviews
Log In
Get Started
HOW IT WORKS
From signup to clarity in 60 seconds
01
Sign in with Google
One tap. No forms, no passwords. Your Google account creates your TYE profile instantly.
02
Log your first expense
Add the amount, pick a category, set a date. Takes less than 10 seconds. Keep going.
03
Set your budgets
Tell TYE how much you want to spend per category. Get alerted before limits are reached.
04
Watch your insights grow
The more you log, the smarter your dashboard gets. Monthly trends, category breakdowns, goal progress.
Recent Transactions
Lunch — Bashundhara
2026-03-14
৳420
Uber to office
2026-03-13
৳250
Clothing — Aarong
2026-03-12
৳2,800
Electricity Bill
2026-03-10
৳950
Netflix Subscription
2026-03-08
৳400

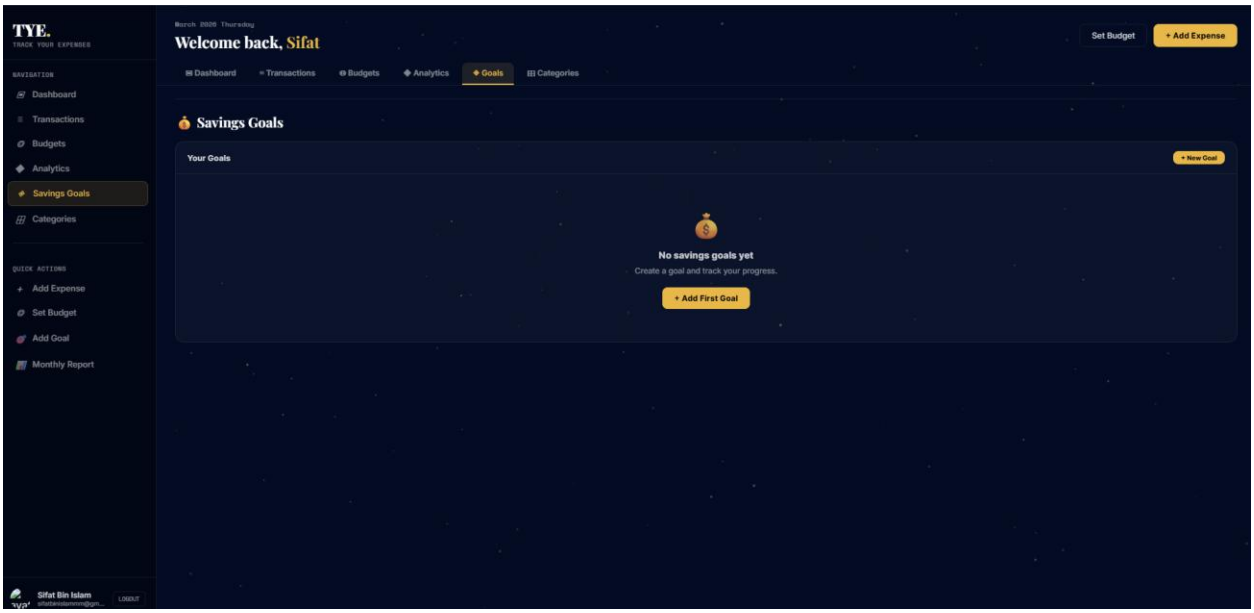

TYE.
Monthly Report
March 2026
৳28,430
Budget Status
All categories
On track
Savings Goal
New Laptop
68%
Your finances, finally clear.
Log expenses in seconds. Understand your habits in minutes. Take control from day one.
1k+
Active users
৳2M+
Tracked monthly
Free
No credit card
"TYE completely changed how I think about money. Impossible to ignore where your spending goes."
— Rafi Ahmed, Freelance Designer
← Back to home
Welcome to TYE
Sign in with Google to access your personal dashboard. Your data stays private.
Google-এর ব্যবহার চালিয়ে যান
Encrypted
OAuth 2.0
GDPR compliant
By continuing, you agree to TYE's Terms and Privacy Policy.

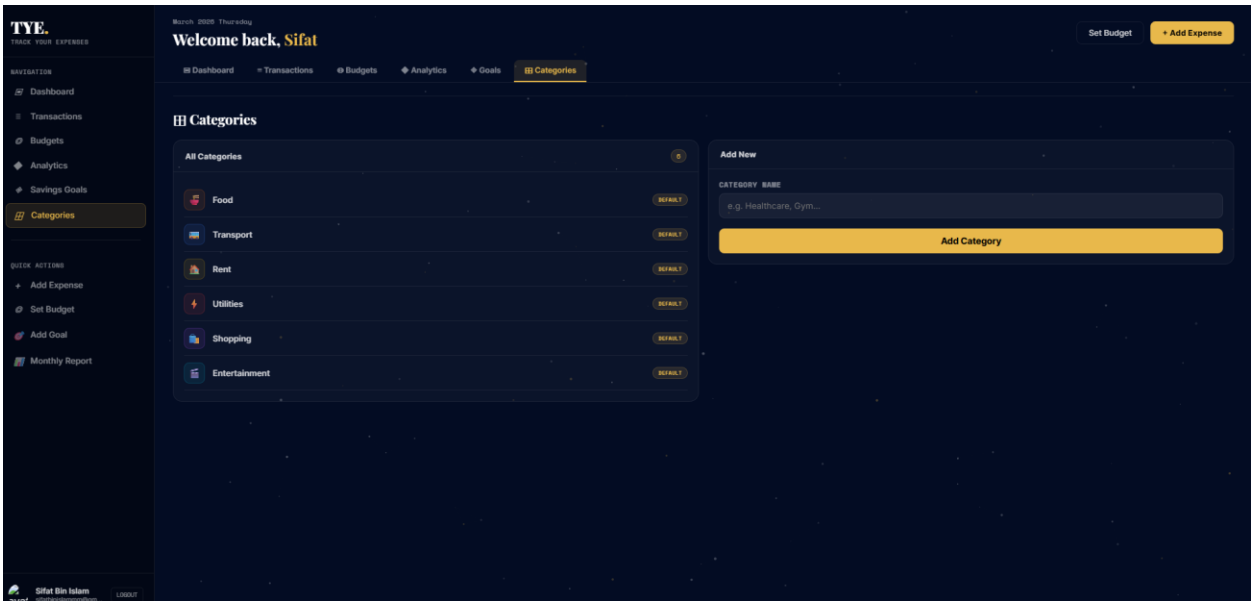

TYE.
TRACK YOUR EXPENSES
NAVIGATION
Dashboard
Transactions
Budgets
Analytics
Savings Goals
Categories
QUICK ACTIONS
Add Expense
Set Budget
Add Goal
Monthly Report
Sifat Bin Islam
LOGOUT
March 2026 Thursday
Welcome back, Sifat
Set Budget
+ Add Expense
Dashboard
Transactions
Budgets
Analytics
Goals
Categories
March 2026
Food, Utilities — budget limit এ পৌঁছে গেছে
TOTAL SPENT
৳14,200
↑111.9% last month
TRANSACTIONS
7
This month
AVG / EXPENSE
৳2029
Per transaction
ACTIVE BUDGETS
5
2 alert
Category Share
Food
Sports
Transport
Utilities
By Category
৳10,000
৳7,500
৳5,000
৳2,500
৳0
Food
Transport
Utilities
Sports
Recent Transactions
7 this month
Food
2026-03-04
৳1,000
Food
2026-03-02
৳3,500
Budget Status — March 2026
Food
100%
৳8,700 spent
৳7,000 limit
Over Budget

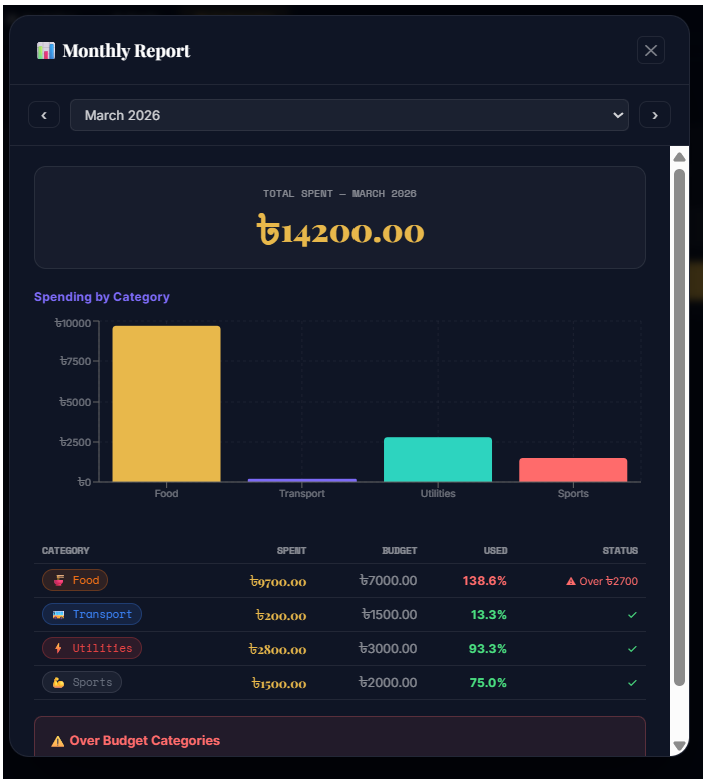

TYE.
TRACK YOUR EXPENSES
NAVIGATION
Dashboard
Transactions
Budgets
Analytics
Savings Goals
Categories
QUICK ACTIONS
Add Expense
Set Budget
Add Goal
Monthly Report
Sifat Bin Islam
LOGOUT
March 2026 Thursday
Welcome back, Sifat
Set Budget
+ Add Expense
Dashboard
Transactions
Budgets
Analytics
Goals
Categories
All Transactions
History
12 total
Food
2026-03-04
৳1,000
Food
2026-03-02
৳3,500
Transport
2026-03-17
৳200
Food
2026-02-24
৳400
Shopping
2026-02-15
৳1,000
Food
2026-03-14
৳2,200
Food
2026-02-03
৳2,000
Food
2026-02-18
৳2,000
Transport
2026-02-04
৳1,300
Utilities
2026-03-16
৳2,800
yumcha district
Food
2026-03-15
৳3,000


TYE.
TRACK YOUR EXPENSES
NAVIGATION
Dashboard
Transactions
Budgets
Analytics
Savings Goals
Categories
QUICK ACTIONS
Add Expense
Set Budget
Add Goal
Monthly Report
Sifat Bin Islam
LOGOUT
March 2026 Thursday
Welcome back, Sifat
Set Budget
+ Add Expense
Dashboard
Transactions
Budgets
Analytics
Goals
Categories
Budget Management
March 2026
All Budgets
Food
100%
৳6,700 spent
৳7,000 limit
Over Budget
Rent
0%
৳0 spent
৳3,000 limit
Sports
75%
৳1,500 spent
৳2,000 limit
Transport
13%
৳200 spent
৳1,500 limit
Utilities
93%
৳2,880 spent
৳3,000 limit
Near Limit
Set / Update Budget
CATEGORY
Food
MONTHLY LIMIT (৳)
5000
ALERT AT (%)
80
Save Budget


TYE.
TRACK YOUR EXPENSES
NAVIGATION
Dashboard
Transactions
Budgets
Analytics
Savings Goals
Categories
QUICK ACTIONS
Add Expense
Set Budget
Add Goal
Monthly Report
Sifat Bin Islam
LOGOUT
March 2026 Thursday
Welcome back, Sifat
Set Budget
+ Add Expense
Dashboard
Transactions
Budgets
Analytics
Goals
Categories
Analytics
6-Month Trend
৳16,000
৳12,000
৳8,000
৳4,000
৳0
Feb
Mar
Monthly Summary
Feb
৳6,700
Mar
৳14,200
+112%


TYE.
TRACK YOUR EXPENSES
NAVIGATION
Dashboard
Transactions
Budgets
Analytics
Savings Goals
Categories
QUICK ACTIONS
Add Expense
Set Budget
Add Goal
Monthly Report
Sifat Bin Islam
LOGOUT
March 2026 Thursday
Welcome back, Sifat
Set Budget
+ Add Expense
Dashboard
Transactions
Budgets
Analytics
Goals
Categories
Savings Goals
Your Goals
+ New Goal
No savings goals yet
Create a goal and track your progress.
+ Add First Goal


TYE.
TRACK YOUR EXPENSES
NAVIGATION
Dashboard
Transactions
Budgets
Analytics
Savings Goals
Categories
QUICK ACTIONS
Add Expense
Set Budget
Add Goal
Monthly Report
Sifat Bin Islam
LOGOUT
March 2026 Thursday
Welcome back, Sifat
Set Budget
+ Add Expense
Dashboard
Transactions
Budgets
Analytics
Goals
Categories
Categories
All Categories
5
Food
DEFAULT
Transport
DEFAULT
Rent
DEFAULT
Utilities
DEFAULT
Shopping
DEFAULT
Entertainment
DEFAULT
Add New
CATEGORY NAME
e.g. Healthcare, Gym...
Add Category


Monthly Report
March 2026
TOTAL SPENT — MARCH 2026
৳14200.00
Spending by Category
৳10000
৳7500
৳5000
৳2500
৳0
Food
Transport
Utilities
Sports
CATEGORY
SPENT
BUDGET
USED
STATUS
Food
৳9700.00
৳7000.00
138.6%
Over ৳2700
Transport
৳200.00
৳1500.00
13.3%
Utilities
৳2800.00
৳3000.00
93.3%
Sports
৳1500.00
৳2000.00
75.0%
Over Budget Categories

Chapter 8: Unit Testing

8.1 Testing Strategy

The testing strategy for TYE covers both the Python backend and the React frontend. Backend tests are implemented using the pytest framework. Frontend tests are implemented using Vitest and the React Testing Library. All tests are designed to execute without reliance on external services: database tests use an in-memory SQLite instance, and Google Cloud Vision API calls are replaced with mock objects. This ensures that the test suite can be executed in any environment without network access or credential configuration.

Tests are categorised as unit tests, which verify the behaviour of individual functions or modules in isolation, and integration tests, which verify the behaviour of API endpoints through the full request-validation-database-response cycle using FastAPI's TestClient.

8.2 Backend Test Coverage

Schema Validation Tests (test_schemas.py)

This file contains 25 tests covering the five Pydantic schema classes. Tests verify that required fields raise ValidationError when absent, that numeric fields accept integer inputs and coerce them to float, that string inputs for numeric fields raise ValidationError, and that optional fields correctly default to their configured values (for example, alert_threshold defaulting to 80.0).

API Route Tests (test_api_routes.py)

This file contains 22 integration tests that exercise the FastAPI endpoints through the TestClient. The test database is an in-memory SQLite instance. Tests cover the health check endpoints, the full transaction CRUD lifecycle including negative and zero amount rejection, the monthly summary endpoint's month filtering logic, and the budget creation duplicate guard.

8.3 Frontend Test Coverage

Component and Hook Tests (App.test.jsx)

This file contains 30 tests covering the App routing logic, LandingPage content and call-to-action buttons, LoginPage authentication flow and error display, and the useApi hook's toYM utility, sessionStorage integration, initial state values, and API call behaviour. Google OAuth components and the Axios library are mocked to prevent real network requests during testing.

8.4 Test Summary

Test File	Test Count	Coverage Area
test_schemas.py	25	Pydantic schema validation, default values, type coercion.
test_api_routes.py	22	FastAPI endpoint integration, CRUD operations, validation rules.
App.test.jsx	30	React routing, component rendering, useApi hook, auth flow.
Total	**77**	**All backend and frontend test cases combined.**

Appendix

A. Project Resources

1. **Project Information Drive:** Google Drive Folder
2. **Source Code Repository:** GitHub — TYE (Track Your Expenses)
3. **Initial Project Proposal:** View Proposal Document